

Systems Final Design Report

In this project I implement two pseudo-task schedulers. The first implementation uses an equal-priority First-In First-Out (FIFO) queue to buffer incoming tasks and execute them as soon as a worker is freed (and resource is available). The second implementation uses a FIFO in addition to a Round Robin scheduling routine which re-queues tasks for further execution later if they are not completed within a time slot (40ms).

FIFO Scheduler

The FIFO scheduler employs a single-threaded task dispatch routine which hangs until the queued task is available for execution. When the task is added to the worker queue, an available worker takes the task and executes it in one burst, then awaits a new task. All workers subscribe to a thread-safe queue which is interfaced by a multi-producer single-consumer channel (using Rust's MPSC container). Resource availability is determined by the scheduler thread where the call stack halts until resource is available, and worker threads are responsible for notifying the scheduler that load has been freed.

In the event of object termination, the scheduler waits for all worker threads to finish working before destroying the main instance.

Round Robin Scheduler

The Round Robin scheduler employs a dual-threaded task dispatch routine which sends tasks to an equal-priority task queue that is popped from and pushed to by an auxiliary dispatch thread. The auxiliary thread is responsible for managing the queue and receives messages from both the scheduler as well as the workers being employed by the scheduler, all tasks being given equal priority regardless of their sender.

Since the worker threads do not stop until the task queue is empty, joining the auxiliary thread is not necessary on the scheduler thread, which simply waits for worker threads to complete in the event of the scheduler's instance termination.

Testing

I keep each scheduler implementation virtually identical and use compile-time macros and environment variables to specify test parameters, so that modifying the source code for tests (or scaling it to multiple instances and implementations) is not necessary. The defaults for the tests employ the runtime parameters required by the project so that running the required tests is straightforward.

Running the tests

- `cargo run --no-default-features --features fifo_pool` runs 1000 tasks with a 70%/30% CPU/IO distribution and uniform arrival times and tests the FIFO scheduler implementation
- `cargo run --no-default-features --features dynamic_pool` runs 1000 tasks with a 70%/30% CPU/IO distribution and uniform arrival times and tests the Round Robin scheduler implementation

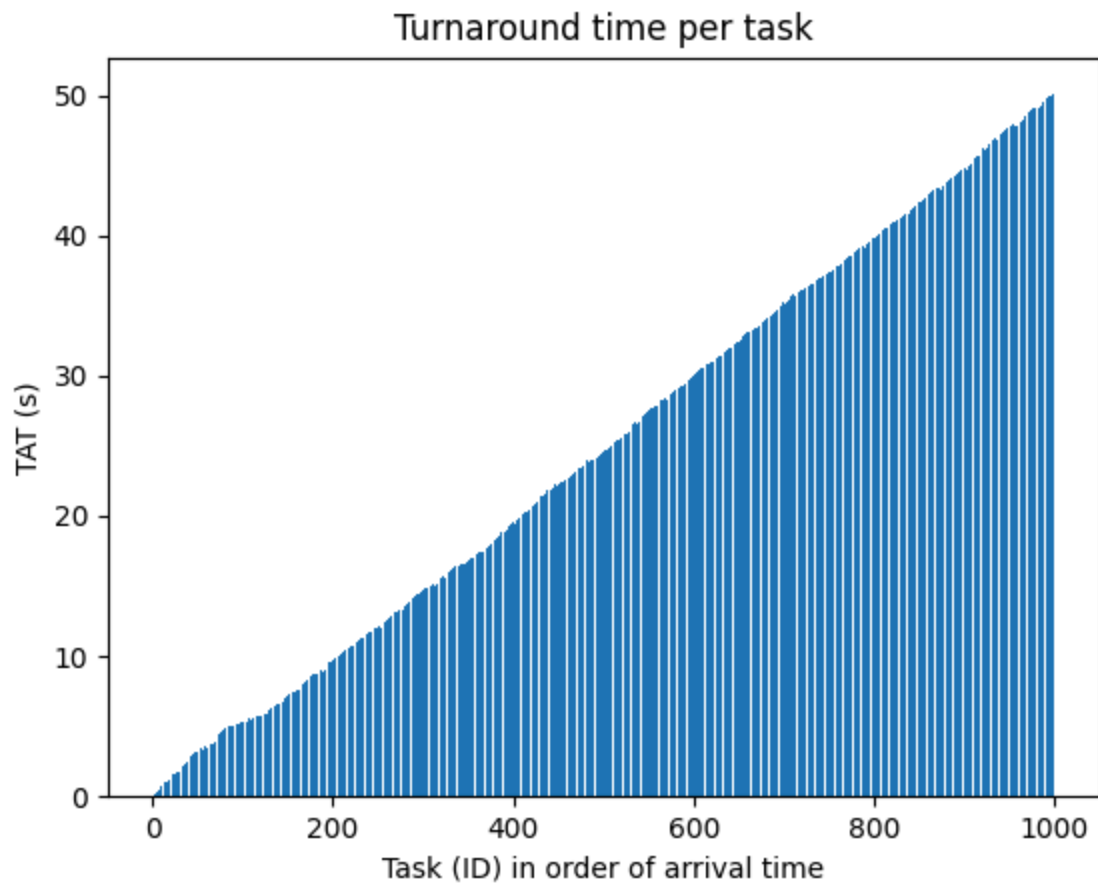
To generate all tests and result summaries automatically, you may run `run_tests.py` (see the README).

Results

Executing 1000 tasks -- each with uniform execution time -- both the FIFO scheduler and the Round Robin scheduler yield similar makespans (the Round Robin scheduler being slightly slower) and maximum completion time. However, the Round Robin scheduler excels at time-sharing and all tasks yield turnaround times much closer to the average in contrast with the FIFO scheduler, whose average TAT scales with the total number of tasks employed over time. Additionally, tasks dispatched by the Round Robin scheduler see negligible average wait time. The Round Robin scheduler also yields a lower average load, since tasks of all types are queued and re-queued uniformly.

FIFO Results

```
1  Total tasks:           1000
2      (CPU):             695
3      (IO):              305
4  Average wait time:     24.6150s
5  Maximum wait time:     49.8129s
6  Average turnaround:    24.885s
7  Average burst time:    0.270s
8  Makespan:           75.172s
9  Avg. load:             77
```



Round Robin Results

1	Total tasks:	1000
2	(CPU):	695
3	(IO):	305
4	Average wait time:	0.0000s
5	Maximum wait time:	0.0055s
6	Average turnaround:	52.562s
7	Average burst time:	N/A, equivalent to quanta (40ms)
8	Makespan:	81.633s
9	Avg. load:	70

